

Practical: Multiclass Classification using Deep Neural Network (MNIST Handwritten Digit Recognition)

Laboratory Practice V – Computer Engineering

1 Aim

Multiclass classification using Deep Neural Networks: Use the MNIST handwritten digit dataset to classify images of digits (0–9) into their respective categories.

1.1 Objective

Students should be able to solve Multiclass Classification using Deep Neural Networks on the MNIST dataset.

1.2 Outcome

Student can apply the technique of Deep Neural Network for implementing multiclass classification.

2 Theory

2.1 What is Multiclass Classification?

Multiclass classification is a type of **supervised learning** where the goal is to assign input data to one of **three or more** classes or categories. Unlike binary classification (2 classes only), multiclass classification involves several possible output categories.

Each input sample is associated with a **single class label**, and the model learns a function that can accurately predict the correct class for new, unseen data.

2.2 Examples of Multiclass Classification

- **Image classification:** Classify images into categories (animals, vehicles, digits)
- **Text classification:** Classify documents into topics (politics, sports, entertainment)
- **Disease diagnosis:** Classify patients into disease types
- **Speech recognition:** Recognize spoken words in multiple languages
- **Digit recognition:** Classify handwritten digits 0–9 (this practical!)

2.3 How DNN Works for Multiclass Classification

1. **Input Layer** — receives raw data (784 pixel values for 28×28 image)
2. **Hidden Layers** — neurons apply linear transformations + non-linear activation (ReLU) to extract features
3. **Dropout Layers** — randomly drop neurons during training to prevent overfitting
4. **Output Layer** — uses **Softmax** to produce probability distribution over all classes
5. **Loss Function** — Categorical Cross-Entropy measures error between predicted and actual class
6. **Optimizer** — RMSprop adjusts weights to minimize loss
7. **Training** — model iterates over data multiple times (epochs), adjusting weights each time

2.4 MNIST Dataset

Property	Details
Total Images	70,000 grayscale images (28×28 pixels)
Training Set	60,000 images
Test Set	10,000 images
Classes	10 (digits 0–9)
Pixel Values	0 (black) to 255 (white)
Built-in	Available via <code>keras.datasets.mnist</code>

2.5 Key Concepts Used

Concept	Meaning
One-Hot Encoding	Label “5” becomes [0,0,0,0,0,1,0,0,0,0]. Each class gets its own binary column
Categorical Cross-Entropy	Loss function for multiclass classification. Measures how far predicted probabilities are from actual one-hot labels
Softmax	Converts raw outputs to probabilities summing to 1 over all classes
ReLU	$f(x) = \max(0, x)$ — activation for hidden layers
RMSprop	Optimizer that adapts learning rate for each parameter
Dropout	Randomly disables a fraction of neurons during training to reduce overfitting
Batch Size	Number of samples processed before updating weights (128 here)
Epoch	One complete pass through the entire training dataset

3 Code

Listing 1: MNIST Multiclass Classification using DNN

```

1 # =====
2 # Step 1: Import Libraries & Load Dataset
3 # =====
4 import numpy as np
5 from tensorflow.keras.models import Sequential
6 from tensorflow.keras.layers import Dense, Dropout
7 from tensorflow.keras.optimizers import RMSprop
8 from tensorflow.keras.datasets import mnist
9 import matplotlib.pyplot as plt
10 from sklearn import metrics
11
12 (x_train, y_train), (x_test, y_test) = mnist.load_data()
13
14 # =====
15 # Step 2: Visualize a Sample Image
16 # =====
17 plt.imshow(x_train[0], cmap='gray')
18 plt.show()
19 print(x_train[0]) # shows 28x28 pixel array
20
21 # =====
22 # Step 3: Check Data Shapes
23 # =====
24 print("X_train shape", x_train.shape) # (60000, 28, 28)
25 print("y_train shape", y_train.shape) # (60000,)
26 print("X_test shape", x_test.shape) # (10000, 28, 28)
27 print("y_test shape", y_test.shape) # (10000,)
28
29 # =====
30 # Step 4: Preprocess Data
31 # =====
32 x_train = x_train.reshape(60000, 784)
33 x_test = x_test.reshape(10000, 784)
34 x_train = x_train.astype('float32')
35 x_test = x_test.astype('float32')
36 x_train /= 255 # Normalize to [0, 1]
37 x_test /= 255
38
39 # =====
40 # Step 5: One-Hot Encode Labels
41 # =====
42 num_classes = 10
43 y_train = np.eye(num_classes)[y_train]
44 y_test = np.eye(num_classes)[y_test]
45
46 # =====
47 # Step 6: Build DNN Model
48 # =====
49 model = Sequential()
50 model.add(Dense(512, activation='relu', input_shape=(784,)))

```

```
51 model.add(Dropout(0.2))
52 model.add(Dense(512, activation='relu'))
53 model.add(Dropout(0.2))
54 model.add(Dense(num_classes, activation='softmax'))
55
56 # =====
57 # Step 7: Compile the Model
58 # =====
59 model.compile(loss='categorical_crossentropy',
60               optimizer=RMSprop(),
61               metrics=['accuracy'])
62
63 # =====
64 # Step 8: Train the Model
65 # =====
66 batch_size = 128
67 epochs = 20
68 history = model.fit(x_train, y_train,
69                    batch_size=batch_size,
70                    epochs=epochs,
71                    verbose=1,
72                    validation_data=(x_test, y_test))
73
74 # =====
75 # Step 9: Evaluate the Model
76 # =====
77 score = model.evaluate(x_test, y_test, verbose=0)
78 print('Test loss:', score[0])
79 print('Test accuracy:', score[1])
```

4 Output

4.1 Data Shapes

```
X_train shape (60000, 28, 28)
y_train shape (60000,)
X_test shape (10000, 28, 28)
y_test shape (10000,)
```

4.2 Training Output (Selected Epochs)

```
Epoch 1/20 - accuracy: 0.9216 - val_accuracy: 0.9547
Epoch 5/20 - accuracy: 0.9850 - val_accuracy: 0.9790
Epoch 10/20 - accuracy: 0.9930 - val_accuracy: 0.9836
Epoch 15/20 - accuracy: 0.9956 - val_accuracy: 0.9835
Epoch 20/20 - accuracy: 0.9969 - val_accuracy: 0.9843
```

4.3 Final Test Result

```
Test loss: 0.0854
Test accuracy: 0.9852 (98.52%)
```

5 Code Explanation (Hinglish)

5.1 Is Practical Ka Aim Kya Hai?

Humein MNIST dataset ki **70,000 handwritten digit images** (0 se 9) dekhke batana hai ki image mein kaunsa digit hai. Total **10 classes** hain (0, 1, 2, ... 9).

Yeh ek **Multiclass Classification** problem hai — DNN (Deep Neural Network) use karke solve karenge. Binary classification mein sirf 2 classes hote hain, lekin yahan 10 classes hain — isliye “multi”.

5.2 Karna Kya Hai? (Har Step Ka Matlab)

5.2.1 Step 1: Import Libraries & Load Dataset

```
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

- Keras ke built-in function se MNIST dataset directly download hota hai
- `x_train` = 60,000 training images (28×28 pixels each)
- `y_train` = labels (0–9) — kaunsa digit hai
- `x_test` = 10,000 test images
- `y_test` = unke labels

Simple: 70,000 handwritten digit images download kiye.

5.2.2 Step 2: Visualize Sample Image

```
plt.imshow(x_train[0], cmap='gray')
print(x_train[0])
```

Ek sample image dikhai — ek “5” ka handwritten digit dikhta hai. Har image basically ek **28×28 number array** hai jismein 0 = black, 255 = white.

Simple: Dekha ki data kaisa dikhta hai — numbers ka grid jo image banata hai.

5.2.3 Step 3: Check Data Shapes

```
print("X_train shape", x_train.shape) # (60000, 28, 28)
print("y_train shape", y_train.shape) # (60000,)
```

- Training images: 60,000 images, har ek 28×28 pixels
- Labels: 60,000 single numbers (0–9)

Simple: Data ka shape verify kiya.

5.2.4 Step 4: Preprocess Data (Flatten + Normalize)

```
x_train = x_train.reshape(60000, 784) # 28x28 = 784
x_train = x_train.astype('float32')
x_train /= 255 # Normalize to [0, 1]
```

- `reshape(60000, 784)` — 2D image (28×28) ko 1D vector (784) mein flatten kiya. DNN ko flat input chahiye (CNN nahi hai yahan!)
- `astype('float32')` — integer se float mein convert kiya (neural network ke liye)
- `/= 255` — pixel values 0–255 se 0–1 mein normalize kiya

Kyun? Neural network normalized data pe better kaam karta hai. 0–1 range mein training faster aur stable hoti hai.

Simple: Images ko flat 784-length vector mein convert kiya aur 0–1 mein normalize kiya.

5.2.5 Step 5: One-Hot Encode Labels

```
num_classes = 10
y_train = np.eye(num_classes)[y_train]
y_test = np.eye(num_classes)[y_test]
```

- Label “5” → [0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
- Label “0” → [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- Har label ek 10-length vector ban jaata hai jismein sirf ek position pe 1 hota hai
- `np.eye(10)` — 10×10 identity matrix banata hai, phir label se corresponding row utha leta hai

Kyun? Softmax output 10 probabilities deta hai, toh labels bhi 10-length mein chahiye comparison ke liye (`categorical_crossentropy` ke liye zaruri hai).

Simple: Labels ko numbers se one-hot vectors mein badla.

5.2.6 Step 6: Build DNN Model

```
model = Sequential()
model.add(Dense(512, activation='relu', input_shape=(784,)))
model.add(Dropout(0.2))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(num_classes, activation='softmax'))
```

Layer by layer samjho:

1. **Dense(512, relu, input=784)** — 784 pixel values lete hai, 512 neurons se process karte hain. Parameters: $784 \times 512 + 512 = 4,01,920$
2. **Dropout(0.2)** — training mein 20% neurons randomly band — overfitting rokta hai
3. **Dense(512, relu)** — phir 512 neurons ka ek aur hidden layer. Parameters: $512 \times 512 + 512 = 2,62,656$

4. **Dropout(0.2)** — phir se 20% dropout

5. **Dense(10, softmax)** — **10 neurons** for 10 classes. Softmax har class ki probability deta hai (sab milke = 1). Parameters: $512 \times 10 + 10 = 5,130$

Note: Yeh CNN nahi hai — sirf Dense layers hain. Image ko pehle hi flatten kiya (784 vector). Yeh DNN approach hai.

Simple: 784 pixels $\rightarrow$ 512 neurons $\rightarrow$ dropout $\rightarrow$ 512 neurons $\rightarrow$ dropout $\rightarrow$ 10 outputs (kaunsa digit hai).

5.2.7 Step 7: Compile Model

```
model.compile(loss='categorical_crossentropy',
              optimizer=RMSprop(),
              metrics=['accuracy'])
```

- `categorical_crossentropy` — multiclass classification ke liye loss function (one-hot labels ke saath use hota hai)
- `RMSprop()` — optimizer jo weights update karega (Adam ka cousin hai)
- `accuracy` — kitne % sahi classify kiye track karega

Simple: Model ko bataya ki kaise seekhna hai — loss kaise measure karo aur weights kaise update karo.

5.2.8 Step 8: Train Model

```
history = model.fit(x_train, y_train,
                   batch_size=128, epochs=20,
                   validation_data=(x_test, y_test))
```

- `batch_size=128` — ek baar mein 128 images process karega. $60,000/128 = 469$ batches per epoch
- `epochs=20` — poora dataset 20 baar dekhega
- Training: 92% (Epoch 1) $\rightarrow$ 99.7% (Epoch 20)
- Validation: 95.5% $\rightarrow$ 98.4%

Simple: Model ne 20 baar 60,000 images dekhke seekha — 99.7% training accuracy mili!

5.2.9 Step 9: Evaluate

```
score = model.evaluate(x_test, y_test, verbose=0)
print('Test loss:', score[0])      # 0.0854
print('Test accuracy:', score[1])  # 98.52%
```

Final test accuracy: 98.52% — 100 mein se ~99 digits sahi classify kiye!

Simple: Final exam diya — 98.52% marks aaye.

5.3 Final Outcome Kya Milta Hai?

1. **Trained DNN Model** — jo handwritten digit image dekhke 0–9 mein se category bata sakta hai
2. **98.52% Test Accuracy** — 100 mein se ~99 digits sahi!
3. **10-class classifier** — har image ke liye 10 probabilities deta hai
4. **Multiclass approach** — One-Hot encoding + Softmax + Categorical Cross-Entropy

Bottom Line: Is practical mein humne dekha ki kaise raw image data (28×28 pixels) ko flatten karke ek DNN (3 Dense layers) se multiclass classification ki. Model ne 98.52% accuracy achieve ki. Yeh CNN use nahi karta — sirf Dense layers se kaam chalaya, phir bhi bahut acchi accuracy aayi.

6 How to Run

6.1 On Google Colab (Recommended)

1. Go to colab.research.google.com
2. Upload the .ipynb notebook
3. Run all cells — dataset automatically downloads from Keras

6.2 On Fresh Ubuntu PC

1. **Update & install Python:**

```
sudo apt update && sudo apt upgrade -y
sudo apt install python3 python3-pip python3-venv -y
```

2. **Create virtual env & install:**

```
python3 -m venv dl_env
source dl_env/bin/activate
pip install numpy tensorflow matplotlib scikit-learn jupyter
```

3. **Run notebook:**

```
jupyter notebook
# Open .ipynb and run all cells
# MNIST downloads automatically via Keras
```

7 FAQ

1. **What is Batch Size?**

Number of samples processed before the model's weights are updated. Here `batch_size = 128`, so $60,000/128 = 469$ batches per epoch. Larger batches = faster training but more memory.

2. What is Dropout?

Regularization technique that randomly “drops” (disables) a percentage of neurons during training. This forces the network to not rely on specific neurons, reducing overfitting. Dropout(0.2) means 20% neurons are randomly turned off each training step.

3. What is RMSprop?

Root Mean Square Propagation — an adaptive learning rate optimizer. It adjusts the learning rate for each parameter based on the average of recent gradients. Good for non-stationary problems.

4. What is the Softmax Function?

Activation function used in the output layer for multiclass classification. It converts raw output values (logits) into **probabilities** that sum to 1 across all classes. Formula: $\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$

5. What is the ReLU Function?

Rectified Linear Unit: $f(x) = \max(0, x)$. If input is positive, output = input. If negative, output = 0. Simple, fast, and avoids the vanishing gradient problem.

8 Conclusion

In this practical, we implemented Multiclass Classification of handwritten digits using a Deep Neural Network on the MNIST dataset. The 28×28 pixel images were flattened to 784-dimensional vectors, normalized to [0, 1], and labels were one-hot encoded. A 3-layer DNN with Dense(512, ReLU) → Dropout(0.2) → Dense(512, ReLU) → Dropout(0.2) → Dense(10, Softmax) architecture was built. The model was compiled with RMSprop optimizer and Categorical Cross-Entropy loss, trained for 20 epochs with batch size 128. The model achieved a final test accuracy of approximately 98.52%, demonstrating the effectiveness of Deep Neural Networks for multiclass classification tasks.

9 Offline Dataset Loading (No Internet)

Agar lab PC pe internet nahi hai, toh MNIST dataset offline load kar sakte hain. Neeche steps diye hain:

9.1 Download Links

Pehle kisi internet-wale PC se yeh 4 files download karo:

File	URL
train-images-idx3-ubyte.gz	https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz

Single file download (Recommended):

Keras MNIST dataset ek single .npz file mein aata hai:

<https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

Alternative source (Original MNIST):

Yann LeCun ki official site se 4 individual files bhi download kar sakte hain:

```
http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz
http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz
http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz
http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz
```

Kaggle link:

```
https://www.kaggle.com/datasets/hojjatk/mnist-dataset
```

9.2 Method 1: Using Keras Cache (Easiest)

Agar pehle se `mnist.npz` file download kar li hai toh use Keras ke cache folder mein copy karo:

```
# Linux/Ubuntu:
mkdir -p ~/.keras/datasets/
cp mnist.npz ~/.keras/datasets/mnist.npz

# Windows:
# Copy mnist.npz to:
# C:\Users\<YourUsername>\.keras\dasetsets\mnist.npz
```

Ab original code bina kisi change ke kaam karega:

```
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
# Keras will find the file in cache -- no download needed!
```

Simple: Pehle se downloaded file ko Keras ke cache folder mein daal do, phir code normally chalega.

9.3 Method 2: Load from Local .npz File

Agar cache folder mein nahi rakhna chahte, toh direct path se load karo:

```
import numpy as np

# mnist.npz file ko project folder mein rakh lo
path = 'mnist.npz' # ya full path do

with np.load(path, allow_pickle=True) as data:
    x_train = data['x_train']
    y_train = data['y_train']
    x_test = data['x_test']
    y_test = data['y_test']

print("Offline MNIST Loaded!")
print("x_train shape:", x_train.shape) # (60000, 28, 28)
print("y_train shape:", y_train.shape) # (60000,)
```

Baaki ka code same rahega — reshape, normalize, one-hot, model build, train, evaluate — koi change nahi.

9.4 Method 3: Load from 4 .gz Files (Original Format)

Agar Yann LeCun site se 4 individual .gz files download ki hain:

```
import gzip
import numpy as np

def load_mnist_images(filename):
    with gzip.open(filename, 'rb') as f:
        data = np.frombuffer(f.read(), np.uint8, offset=16)
    return data.reshape(-1, 28, 28)

def load_mnist_labels(filename):
    with gzip.open(filename, 'rb') as f:
        data = np.frombuffer(f.read(), np.uint8, offset=8)
    return data

x_train = load_mnist_images('train-images-idx3-ubyte.gz')
y_train = load_mnist_labels('train-labels-idx1-ubyte.gz')
x_test  = load_mnist_images('t10k-images-idx3-ubyte.gz')
y_test  = load_mnist_labels('t10k-labels-idx1-ubyte.gz')

print("MNIST loaded from .gz files!")
print("x_train:", x_train.shape)  # (60000, 28, 28)
```

9.5 Folder Structure (Offline Setup)

```
project_folder/
|-- Demo_Mnist.ipynb          (notebook file)
|-- mnist.npz                 (Method 2)
|   OR
|-- train-images-idx3-ubyte.gz (Method 3)
|-- train-labels-idx1-ubyte.gz
|-- t10k-images-idx3-ubyte.gz
|-- t10k-labels-idx1-ubyte.gz
```

Summary: Sabse aasan tarika hai Method 1 (Keras cache) — sirf `mnist.npz` file download karke `.keras/datasets/` folder mein copy karo, phir code mein koi change ki zarurat nahi.